

What is an Artificial Neural Network?

An **Artificial Neural Network (ANN)** is a machine learning model inspired by the way the human brain processes information.

Just like our brain consists of billions of neurons connected together, an ANN consists of many artificial neurons connected in layers.

These neurons work together to solve problems such as:

- Image recognition
- Speech recognition
- Language translation
- Medical diagnosis
- Stock prediction
- Fraud detection

Structure of an ANN

An ANN consists of three types of layers.

- Input Layer
- Hidden Layer
- Output Layer

1. Input Layer

The input layer receives the data.

Example

Predicting house price

Inputs

- Size
- Bedrooms
- Age
- Location score

Nothing happens here except passing the data.

2. Hidden Layer

This is where learning happens.

The hidden layer:

- finds patterns
- performs calculations
- extracts important features

Example

For face recognition

Input image



Hidden layer learns

- Eyes
- Nose
- Mouth
- Shape



Another hidden layer learns

- Complete face

3. Output Layer

Produces the final prediction.

Example

Cat image

Output

Cat : 0.98

Dog : 0.02

Prediction = Cat

Artificial Neuron

One neuron performs three steps.

Inputs

x_1

x_2

x_3

↓

Multiply with weights

↓

Add bias

↓

Activation Function

↓

Output

Mathematically

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Then

$$Output = Activation(z)$$

What are Weights?

Weights tell us how important an input is.

Example

House Price

Features

- Size
- Age
- Number of Bedrooms

Suppose

Weight(Size) = 0.9

Weight(Age) = 0.2

Weight(Bedrooms) = 0.5

The network learns that **size has the greatest influence** on house price.

What is Bias?

Bias is an extra value added to a neuron that helps it make better decisions.

Think of bias as a **fine-tuning knob** that shifts the neuron's output.

The equation of a neuron is:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

where:

- x = Inputs
- w = Weights
- b = Bias
- z = Weighted sum

Then an activation function is applied to z .

Example:

Suppose a student needs **40 marks** to pass.

Imagine the ANN predicts marks based on assignments and attendance.

Without bias:

Marks = $5 \times (\text{Assignments})$

If Assignments = 0

Marks = 0

But what if every student automatically receives **5 grace marks**?

Then the equation becomes

Marks = $5 \times (\text{Assignments}) + 5$

That +5 is like the bias.

Bias shifts the result upward.

- **Weights** answer the question: "**How important is each input?**"
- **Bias** answers the question: "**Should the neuron's output be shifted up or down regardless of the inputs?**"

Together, they make the neuron flexible enough to learn real-world patterns.

Weights decide the importance of each input, while bias is an extra adjustable value that shifts the neuron's output so it can learn more accurate decision boundaries.

What is an Activation Function?

An **activation function** decides **whether a neuron should be activated (send information to the next layer) or not**.

Think of it as an **ON/OFF switch** or a **decision-maker**.

The neuron first calculates:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Then this value is passed through an activation function.

Types of Activation Functions

There are many activation functions, but the most commonly used ones are:

1. Binary Step Function
2. Linear Activation Function
3. Sigmoid Function
4. Tanh Function
5. ReLU (Rectified Linear Unit)
6. Leaky ReLU
7. ELU
8. Softmax

1. Binary Step Function

This is the simplest activation function.

Rule

If input ≥ 0

Output =1

Otherwise

Output =0

Formula

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Advantages

- Very simple
- Fast

Disadvantages

- Only gives Yes or No
- Cannot learn complex problems
- Not used in deep learning today

2. Linear Activation Function

Formula

$$f(x) = x$$

Output equals input.

Example

Input =15

Output =15

Nothing changes.

Why isn't it used in hidden layers?

Suppose you have

100 hidden layers.

Each uses

$f(x)=x$

Then the network still behaves like a straight line.

It cannot solve non-linear problems.

Mostly used in **output layers for regression problems** (e.g., predicting house prices).

3. Sigmoid Function

This is one of the most famous activation functions.

Formula

$$f(x) = \frac{1}{1 + e^{-x}}$$

It converts any number into a value between **0 and 1**.

Think of it as converting raw scores into probabilities.

Example

Email Spam Detection

Output =0.95

Meaning

95% chance the email is spam.

Advantages

- Produces probabilities
- Smooth curve
- Good for binary classification output

Disadvantages

- Slow compared to ReLU
- Suffers from the **vanishing gradient problem** (gradients become very small during training)

4. Tanh (Hyperbolic Tangent)

Formula

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Output ranges from -1 to +1.

Why use Tanh?

Unlike Sigmoid, Tanh is centered around zero, which often helps the network learn faster.

Applications

- Older recurrent neural networks (RNNs)
- Hidden layers in some architectures

5. ReLU (Rectified Linear Unit)

This is the **most widely used activation function** in modern deep learning.

Formula

$$f(x) = \max(0, x)$$

Rule

If input is negative

Output =0

Otherwise

Output =Input

Negative values are ignored.

Advantages

- Very fast
- Easy to compute
- Trains deep networks efficiently

- Reduces vanishing gradient issues

Disadvantage

Some neurons may always output 0 and stop learning (called the **Dying ReLU** problem).

6. Leaky ReLU

Leaky ReLU solves the Dying ReLU problem.

Formula

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x < 0 \end{cases}$$

Instead of making all negative values zero, it allows a small negative output.

Example

Input = -10

Output = -0.1

Instead of

0

the neuron still receives a tiny signal and can continue learning.

7. ELU (Exponential Linear Unit)

ELU is another improvement over ReLU.

- Positive values behave like ReLU.
- Negative values become small negative numbers instead of zero.

Advantages

- Faster convergence
- Better learning in some deep networks

Disadvantage

- More computationally expensive than ReLU

8. Softmax

Softmax is used for **multi-class classification**.

Instead of giving one output, it assigns probabilities to all classes.

Example

Image Recognition

Input

Image of an animal

Output

Class	Probability
-------	-------------

Cat	0.80
-----	------

Dog	0.15
-----	------

Horse	0.03
-------	------

Bird	0.02
------	------

All probabilities add up to **1.0**.

Prediction

Cat

Applications

- Digit recognition (0–9)
- Animal classification
- Disease classification
- Language translation

Activation Function	Output Range	Common Use	Advantages	Disadvantages
Binary Step	0 or 1	Rarely used	Very simple	Cannot learn complex patterns
Linear	$(-\infty)$ to $(+\infty)$	Regression output	Simple	Cannot model non-linear relationships

Activation Function	Output Range	Common Use	Advantages	Disadvantages
Sigmoid	0 to 1	Binary classification output	Probability interpretation	Vanishing gradient
Tanh	-1 to 1	Some hidden layers	Zero-centered	Vanishing gradient
ReLU	0 to (+∞)	Hidden layers	Fast, efficient	Dying ReLU
Leaky ReLU	Small negative to (+∞)	Hidden layers	Solves Dying ReLU	Slope must be chosen
ELU	Negative values to (+∞)	Hidden layers	Smooth, faster convergence	More computationally expensive
Softmax	0 to 1 (sum = 1)	Multi-class output	Gives class probabilities	Only used in output layer

Which Activation Function To Use?

Problem	Recommended Activation Function
House price prediction (Regression)	Linear (Output Layer)
Spam vs Not Spam	Sigmoid (Output Layer)
Cat vs Dog classification	Sigmoid (Output Layer)
Digit recognition (0–9)	Softmax (Output Layer)
Image classification (Hidden Layers)	ReLU
Deep Neural Networks	ReLU or Leaky ReLU

- An **activation function** decides how much of a neuron's calculated value should be passed to the next layer.
- It introduces **non-linearity**, enabling neural networks to solve complex problems like image recognition and language translation.
- **ReLU** is the default choice for most hidden layers because it is simple and efficient.
- **Sigmoid** is commonly used for binary classification outputs.
- **Softmax** is used for multi-class classification outputs.
- **Linear** activation is typically used for regression outputs.

How many neurons to add to each layer:

There is **no fixed rule** for deciding how many neurons to add to each layer.

General Guidelines

1. Input Layer

The number of neurons equals the number of input features.

Example:

If your dataset has:

- Age
- Salary
- Experience
- Education

Then:

Input neurons = 4

2. Hidden Layer(s)

The number of neurons depends on the complexity of the problem.

Some common guidelines are:

- Start with a number **between the input size and output size**.
- A common starting point is **32, 64, or 128 neurons**.
- For small datasets, use fewer neurons to avoid overfitting.
- For complex problems (such as image recognition), use more neurons and multiple hidden layers.

Example 1: Student Pass/Fail Prediction

Inputs = 5

Output = 1

A reasonable architecture:

Input Layer	: 5 neurons
Hidden Layer	: 8 neurons
Output Layer	: 1 neuron

Example 2: House Price Prediction

Inputs = 10

Output = 1

Possible architecture:

$10 \rightarrow 16 \rightarrow 8 \rightarrow 1$

Example 3: Image Classification

Inputs = 784 (28×28 pixels)

Possible architecture:

$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$

(10 outputs represent digits 0–9.)

3. Output Layer

The number of neurons depends on the prediction task.

Problem	Output Neurons
House price prediction	1
Pass/Fail	1
Spam/Not Spam	1
Cat/Dog/Bird	3
Digit Recognition (0–9)	10